

Dr. Ranjith Shepur

Kubernetes Pocket Guide

A Complete Reference for Container Orchestration

Architecture

Core Components & Design

Networking

Services, Ingress, Network Policy

Security

RBAC, Auth, Audit, Certs

Helm & Kustomize

Package Management

Workloads

Pods, Deployments, DaemonSets

Storage

PV, PVC, StatefulSets

Operations

Scaling, Backup, Upgrades

API Resources

All 60+ Resource Types

● About the Author

Dr. Ranjith Shepur

Technology Professional • DevOps & DevSecOps Expert • Cloud Infrastructure Architect

Dr. Ranjith Shepur is a seasoned technology professional with vast experience in DevOps and DevSecOps, bringing together deep expertise in software engineering, security, and cloud infrastructure. With a distinguished career spanning multiple domains, Dr. Shepur has contributed to mission-critical projects across diverse industries.

Key Areas of Expertise

DevOps & DevSecOps Engineering — Architecting CI/CD pipelines, container orchestration, and automated security workflows.

Satellite Project Development — Contributing to high-reliability systems engineering and mission-critical software platforms.

Enterprise Resource Planning (ERP) — Designing and implementing large-scale ERP solutions for complex business processes.

Customer Relationship Management (CRM) — Building CRM systems that enhance customer engagement and retention.

E-Commerce Platforms — Developing scalable, secure online retail solutions from architecture to deployment.

Banking Software & SDLC — Delivering secure, compliant software solutions across the full Software Development Life Cycle for banking institutions.

Dr. Shepur's hands-on experience across satellite systems, ERP, CRM, e-commerce, and banking software has given him a unique perspective on how Kubernetes and cloud-native technologies are applied in real-world, production-grade environments. This book is a distillation of those practical insights, designed to empower the next generation of DevOps practitioners and cloud engineers.

Table of Contents

Foundations		
Kubernetes (k8s)		5
Kubernetes Architecture		7
Methods of Building a Kubernetes Cluster		9
Kubectl & Kubeconfig		
Workloads & Scheduling		
Pods		11
Workloads		13
ReplicaSet & Deployment		
Namespace		15
Resource Requirements & Limits		
Services		17
How Scheduling Works		19
Labels & Selectors		
Annotations		
Node Selector		
Affinity & Anti-Affinity		21
Taints & Tolerations		
Priority Class & Preemption		23
DaemonSet		25
Static Pod		
Autoscaling -- HPA & VPA		27
Configuration & Lifecycle		
ConfigMap & Secrets		29
Init Container		31
Pod Lifecycle		
Sidecar Container & Jobs		33
Rollout & Rollback		
Probes (Self-Healing)		35
Cluster Operations		
Node Maintenance		37
Cluster Upgrade (kubeadm)		
Security		

Authentication & Authorization		39
RBAC -- Role-Based Access Control		
Service Account		41
API Groups		43
Authentication with X.509 (Creating a User)		45
Auditing		47
Network Policy		
Security Context		
Storage		
Storage in Kubernetes		49
Persistent Volumes (PV) & Persistent Volume Claims (PVC)		
StatefulSet		51
Headless Service		
Advanced Topics		
Kustomize		53
Helm		
Operators		
Ingress		55
Cert-Manager		57
Backup & Restore		59
Complete API Reference		
Additional API Resource Types		63
Quick Reference		
Quick Reference -- kubectl Cheat Sheet		61

● How to Use This Guide

Structure -- Organized from foundational concepts to advanced topics. Each section is self-contained with explanations, YAML manifests, and kubectl commands.

Code Blocks -- Dark-background blocks contain real kubectl commands and YAML. Lines starting with \$ are shell commands; indented lines with colons are YAML.

Color Boxes -- Blue = explanations, Green = best practices, Orange = warnings, Red = danger/errors, Yellow = important notes.

Term Labels -- Dark bars with yellow text mark subsections for fast navigation.

Quick Reference -- The final section is a kubectl cheat sheet for daily use.

API Reference -- The penultimate section covers all 60+ Kubernetes API resource types including newer resources like DRA, APF, ValidatingAdmissionPolicy, and more.

Compatibility: All YAML examples use Kubernetes v1.28+ API versions. Commands are tested on kubeadm clusters and managed services (EKS, GKE, AKS).

● Kubernetes (k8s)

Kubernetes (k8s) is an open-source platform for managing containerized workloads and services. It was originally developed by Google and is now maintained by the Cloud Native Computing Foundation (CNCF). Kubernetes lets you declaratively define your application's desired state using objects like pods, services, deployments, configmaps, and more -- then takes care of actually running them.

Why Kubernetes?

Simplify Container Management -- Unified API to deploy and manage containers across multiple hosts or cloud providers.

Enhance Resiliency -- Built-in fault tolerance and self-healing. Keeps apps running despite hardware/software failures.

Improve Scalability -- Scale containerized apps up or down based on demand without downtime.

Increase Automation -- Automates rolling updates, scaling, and load balancing -- reducing ops burden.

Key Interfaces

CRI -- Container Runtime Interface -- Standardized interface between k8s and the container runtime (e.g., containerd, CRI-O). Abstracts runtime details, enabling different runtimes per node.

CNI -- Container Network Interface -- Standard for configuring Linux container network interfaces. CNI plugins set up IP addresses, routing, and network policies for pods.

CSI -- Container Storage Interface -- Standard for exposing storage systems to container orchestrators. Enables dynamic provisioning, deletion, and resizing of storage volumes.

Cluster Components

Kubernetes Cluster MASTER NODE kube-apiserver etcd kube-scheduler controller-manager WORKER NODE kubelet kube-proxy container runtime pods

● Kubernetes Architecture

Kubernetes is built on a master-worker architecture. The master node manages cluster state; worker nodes run actual application workloads.

■ **kube-apiserver** -- The control plane component and primary management entity. Handles all communication and authentication, controls all other components, monitors cluster state. All `kubectl` commands go through here.

■ **etcd** -- Distributed key-value database storing all cluster state data. Only component that directly talks to kube-apiserver. Provides reliable, highly available data store -- cluster can recover quickly from failures.

■ **kube-scheduler** -- Assigns newly created pods to nodes. Reads unassigned pods from etcd, determines best node using algorithms (resource requirements, node capacity, taints, affinity), then informs kube-apiserver.

■ **kube-controller-manager** -- Collection of controllers: Node controller (watches node health), Replication controller, Endpoint controller, ServiceAccount/Token controllers.

■ **kubelet** -- Primary node agent on each worker node. Manages and monitors container state, ensures containers are healthy, communicates with kube-apiserver, manages networking and storage configs.

■ **kube-proxy** -- Manages networking and routing for services. Generates iptables rules on each node so traffic is accurately directed to the appropriate pods -- making service communication reliable.

■ **Container Runtime** -- Runs containers on each node. Manages container lifecycle: pulling images, creating/starting containers, monitoring health, stopping them when no longer needed. (e.g., containerd, CRI-O)

Note: Components needing system resource access (kubelet, kube-proxy) run as system-level services. Others (API server, etcd, scheduler) run as containers in pods.

● Methods of Building a Kubernetes Cluster

Self-Hosted -- Set up and manage your own cluster. Tools: kubeadm kops Rancher kubespray. Good for specific security/compliance needs. Requires deep k8s expertise.

Cloud-Hosted (Managed) -- Cloud providers manage the control plane. You manage worker nodes. Amazon EKS GKE AKS. More cost-effective, less operational overhead.

Cluster as a Service (CaaS) -- Fully managed k8s without worrying about infrastructure. DigitalOcean Linode Platform9.

Containerized / Local -- Run k8s as containerized app -- great for dev/testing. Minikube KinD. Spin up clusters quickly and easily.

kubectl is the command-line tool for managing Kubernetes clusters. It sends HTTP requests to the Kubernetes API server using the API endpoint in the kubeconfig file.

k8s uses a kubeconfig file to store connection info about clusters, users, and contexts. Default location: ~/.kube/config

```
# View current kubeconfig
$ kubectl config view

# Use a specific kubeconfig
$ kubectl --kubeconfig=/etc/kubernetes/admin.conf get node

# Copy admin config to home dir
$ mkdir -p $HOME/.kube
$ sudo scp user@cluster-ip:/etc/kubernetes/admin.conf $HOME/.kube/config

# Switch context (use kubectx)
$ kubectl config use-context arye@k8s-st1

# Autocompletion (zsh)
$ kubectl completion zsh > ~/.oh-my-zsh/custom/plugins/kubectl/kubectl.plugin.zsh
```

Kubeconfig Structure


```
apiVersion: v1
kind: Config
clusters:
  - name: k8s-st1
cluster:
  certificate-authority-data: <cert data>
server: https://127.0.0.1:41285
users:
  - name: arye
user:
  client-certificate-data: <cert data>
client-key-data: <key data>
contexts:
  - name: arye@k8s-st1
context:
  cluster: k8s-st1
  user: arye
namespace: dev
current-context: arye@k8s-st1
```

■■ Access to the cluster config file allows an attacker to view, modify, or delete resources. Restrict access to only those who need it.

● Pods

Pods are the smallest deployable units in Kubernetes. A Pod is a group of one or more containers sharing storage, network resources, and a specification for how to run. Containers cannot be brought up directly without being part of a pod.

Pod YAML Manifest

```
apiVersion: v1 # k8s API version
kind: Pod # resource type
metadata:
  name: pod-nginx
namespace: default
labels:
  app: nginx
type: front-end
spec:
  containers:
    - name: nginx-container
      image: nginx:1.18
```

Common Commands

```
$ kubectl create -f pod-df.yml
$ kubectl get pods -o wide -w
$ kubectl describe pods pod-nginx
$ kubectl delete -f pod-df.yml
$ kubectl explain pods.spec.containers
```

Tip: Use `kubectl apply` instead of `kubectl create` -- it creates or updates resources declaratively.

Process of Creating a Pod

Step 1: Define Pod spec -- Create YAML with containers, labels, volumes, etc.

Step 2: Authentication & Authorization -- kube-api authenticates the request and checks permissions.

Step 3: Manifest Syntax Check -- kube-api validates the YAML against the API schema.

Step 4: Writing to etcd -- Pod manifest is stored in etcd.

Step 5: Pod Scheduling -- Scheduler watches etcd for unscheduled pods, assigns a node.

Step 6: Reporting to API -- Scheduler reports selected node back to kube-api; etcd is updated.

Step 7: Kubelet creates Pod -- Kubelet pulls images, starts containers, updates pod status.

Pod Updates: Only certain fields can be updated on a running pod (e.g., container image). To change other fields (like labels, nodeSelector), you must delete and recreate the pod.

● Workloads

A Workload object defines how to run a containerized application. They abstract away Pod management, handling deployment, scaling, and updates automatically.

Deployment -- Rolling updates & rollbacks. Manages ReplicaSets.

ReplicaSet -- Ensures desired number of pod replicas at all times.

StatefulSet -- Handles stateful apps with stable identities & storage.

DaemonSet -- Runs one pod per node. Used for monitoring/logging agents.

Job -- Creates pods for one-time batch tasks. Tracks completions.

CronJob -- Schedules Jobs on a cron schedule (Unix cron syntax).

HPA -- Horizontal Pod Autoscaler. Scales replicas based on metrics.

VPA -- Vertical Pod Autoscaler. Adjusts CPU/memory requests & limits.

ReplicaSet ensures a specified number of pod replicas run at all times. If a pod fails or is deleted, ReplicaSet automatically creates a replacement.

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: replicaset-nginx
spec:
  replicas: 3
selector:
  matchLabels:
    app: nginx
template:
  metadata:
    labels:
      app: nginx
  spec:
    containers:
      - name: nginx
        image: nginx:1.17
```

```
$ kubectl scale replicaset=3 rs.def.yml --replicas=6
$ kubectl describe rs replicaset-nginx
```

Deployment is a higher-level abstraction over ReplicaSets. Adds rolling updates, rollbacks, and update history. Recommended for stateless applications.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
namespace: dev
spec:
  replicas: 3
selector:
  matchLabels:
    app: nginx
strategy:
  type: RollingUpdate
template:
  metadata:
    labels:
      app: nginx
  spec:
    containers:
      - name: nginx
image: nginx:1.17
```

● Namespace

A Namespace is a virtual cluster within a physical cluster. It provides isolation, resource quotas, and separate DNS for different teams or applications.

```
$ kubectl create namespace dev
$ kubectl get pods -n dev
$ kubectl get pods --all-namespaces
```

ResourceQuota

Limits the total amount of compute resources (CPU, memory) that can be consumed by a set of pods in a namespace.

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: saas-team-quota
namespace: dev
spec:
  hard:
    pods: "10"
  requests.cpu: "2"
  requests.memory: 2Gi
  limits.cpu: "4"
  limits.memory: 4Gi
```

LimitRange

Sets default and maximum resource limits for individual pods/containers in a namespace. Only affects newly created pods.

```
apiVersion: v1
kind: LimitRange
metadata:
  name: dev-resource-limits
namespace: dev
spec:
  limits:
    - default:
        cpu: 100m
      memory: 128Mi
    defaultRequest:
      cpu: 50m
      memory: 64Mi
    max:
      cpu: 500m
      memory: 512Mi
  type: Container
```

Feature	ResourceQuota	LimitRange
Scope	Namespace total	Per pod/container

Feature	ResourceQuota	LimitRange
Purpose	Hard namespace caps	Default & per-item caps
Affects existing pods	No	No

Resource requests = minimum CPU/memory needed to schedule. Resource limits = maximum CPU/memory the container can use.

```
resources:
  requests:
    cpu: "100m" # 0.1 CPU (milliCPU)
    memory: "10M" # 10 megabytes
  limits:
    cpu: "200m" # 0.2 CPU max
    memory: "50M" # 50M max
```

OOM Killer: When a container exceeds its memory limit, the Linux kernel invokes the Out-of-Memory (OOM) Killer to terminate the process inside the container that triggered the OOM condition.

Default values (if not set): CPU request = 0.5, Memory request = 256Mi, CPU limit = 1, Memory limit = 256Mi.

● Services

Services provide a stable IP address and DNS name for a set of pods. They enable load balancing, service discovery, and traffic management within and outside the cluster. When created, a service is assigned a virtual IP (ClusterIP) and DNS name.

ClusterIP Default -- Exposes the service on a cluster-internal IP. Only accessible within the cluster.

type: ClusterIP ports: - targetPort: 80 port: 80

NodePort -- Exposes service on a static port on each node. Accessible via NodeIP:NodePort from outside.

type: NodePort ports: - targetPort: 8080 port: 80 nodePort: 30080

LoadBalancer -- Uses a cloud provider's load balancer. Distributes traffic to pods. Creates NodePort and ClusterIP automatically.

type: LoadBalancer ports: - targetPort: 8080 port: 8080

ExternalName -- Maps the service to an external DNS name. Allows service to be accessed via a consistent name from within the cluster.

type: ExternalName externalName: my.db.example.com

Complete Service Example

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-internal
namespace: dev
spec:
  type: ClusterIP
selector:
  app: nginx # matches pod labels
ports:
  - targetPort: 80 # pod port
    port: 80 # service port
    protocol: TCP
```

Endpoints & DNS

Endpoints: When a Service is created, k8s creates an Endpoint object tracking the IP:port of matching pods. kube-proxy uses this to route traffic.

DNS format: Service: <svc-name>.<namespace>.svc.cluster.local Pod: <pod-ip>.<namespace>.pod.cluster.local

The default DNS provider is CoreDNS, running as pods in kube-system namespace. It retrieves pod/service info from the Kubernetes API.

● How Scheduling Works

When a Pod is created it's marked "unscheduled" and added to a scheduling queue. The scheduler continuously watches this queue and selects the best node using several rules:

Resource Requirements Checks CPU/memory requests vs. node capacity.

Node Capacity Considers total available CPU, memory, storage on each node.

Node Selectors Labels applied to nodes; pods can filter by these labels.

Affinity / Anti-Affinity Allows pods to prefer or avoid certain nodes or co-location with other pods.

Labels are key-value pairs attached to objects (pods, services, nodes). Used for grouping, selecting, and organizing resources. Selectors filter objects based on their labels.

```
# Label a node
$ kubectl label node kubeworker-2 disktype=ssd

# Filter pods by label
$ kubectl get pods -l app=nginx

# Remove a label
$ kubectl label node kubeworker-2 disktype-
```

Annotations are similar to labels but designed to store additional non-identifying metadata (version numbers, timestamps, config details). Up to 256KB in size (labels limited to 63 chars).

```
annotations:
  nginx.ingress.kubernetes.io/proxy-cache: "on"
  nginx.ingress.kubernetes.io/proxy-cache-path: "/data/nginx/cache"
```

NodeSelector lets you specify labels a node must have for a pod to be scheduled there. Simple but limited -- for complex constraints use Affinity.

```
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
spec:
  containers:
    - name: data-processor
  image: data-processor
  nodeSelector:
    disktype: ssd # node must have this label
```

■ ■ If a pod's NodeSelector specifies labels that don't exist on any node, the pod stays in Pending state until a matching node is labeled.

● Affinity & Anti-Affinity

Type	Description
Node Affinity	Rules for which nodes a pod can be scheduled on, based on node labels.
Pod Affinity	Rules to co-locate pods on same node (based on labels of other pods running there).
Pod Anti-Affinity	Rules to prevent pods from being co-located on same node.

Category	Description
requiredDuringSchedulingIgnoredDuringExecution	Rule MUST be satisfied for the pod to be scheduled. If not satisfied, pod won't be scheduled.
preferredDuringSchedulingIgnoredDuringExecution	Rule SHOULD be satisfied but is not required. Pod still schedules if unmet.

Node Affinity Example

```
affinity:
  nodeAffinity:
    preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 80
  preference:
    matchExpressions:
      - key: avail-zone
operator: In
values:
  - zone1
  - weight: 20
preference:
  matchExpressions:
    - key: share
operator: In
values:
  - dedicated
```

Weight values (1–100) determine scheduling priority. Higher weight = higher priority during node selection.

Pod Affinity Example

```
affinity:
  podAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      - labelSelector:
          matchExpressions:
            - key: app
operator: In
values:
  - backend
topologyKey: "rack"
```

Frontend pod MUST be scheduled on a node in the same rack as the backend pod. If no matching node exists, the frontend pod won't be scheduled.

Operators available: In, NotIn, Exists, DoesNotExist

Taints are applied to nodes to repel certain pods. Tolerations are applied to pods to allow them to be scheduled on tainted nodes.

```
# Apply a taint to a node
$ kubectl taint nodes workernode-3 app=ssd:NoSchedule

# Remove a taint
$ kubectl taint nodes workernode-3 app=ssd:NoSchedule-
```

Effect	Behavior
NoSchedule	No new pods scheduled unless they have matching toleration. Existing pods continue.
PreferNoSchedule	Prefer not to schedule here, but will if no better node exists.
NoExecute	Evicts existing pods that don't have a matching toleration.

Toleration in Pod Spec

```
tolerations:
- key: "app"
  operator: "Equal"
  value: "ssd"
  effect: "NoSchedule"
```

Important: Taints alone don't guarantee a pod lands on a specific node -- they only PREVENT certain pods. To guarantee scheduling on a specific node, combine taints with Node Affinity.

● Priority Class & Preemption

Priority Class assigns a priority value (0–1,000,000,000) to a Pod. Higher value = more important. Used for preemption -- evicting lower-priority pods to make room.

```
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: high-priority
value: 1000000
globalDefault: false
preemptionPolicy: PreemptLowerPriority
```

Policy	Behavior
PreemptLowerPriority	Default. Can preempt lower-priority pods to get scheduled.
IfNotOtherPods	Only preempts if no other pods in the cluster can be evicted.
Never	Never preempts. Waits for resources to free up naturally.

Pod Disruption Budget (PDB)

Ensures a certain number or percentage of pods are not voluntarily evicted at the same time -- maintaining high availability during upgrades/maintenance.

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: my-pdb
spec:
  minAvailable: 2 # at least 2 pods always up
  selector:
    matchLabels:
      app: my-app
```

Bin Packing

k8s scheduler uses two strategies to maximize resource usage:

BestFit: Place pod on node with LEAST free resources after placement. Fills nodes up maximally, leaving other nodes empty.

WorstFit: Place pod on node with MOST free resources after placement. Spreads load across nodes.

● DaemonSet

A DaemonSet ensures that all (or some) nodes run a copy of a specific pod. When a new node is added, a pod is automatically created. When a node is removed, the pod is garbage collected. Used for: log collection, monitoring, storage daemons.

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: monitoring-daemon
spec:
  selector:
    matchLabels:
      app: monitoring
  template:
    metadata:
      labels:
        app: monitoring
    spec:
      containers:
        - name: monitoring-agent
      image: monitoring-agent
      nodeSelector:
        Drive: ssd # only on SSD nodes
```

A Static Pod is managed directly by kubelet on a specific node, not by the kube-apiserver. Defined by a YAML manifest placed in a specific directory that kubelet monitors.

Control plane components (API server, controller manager, scheduler) are started as Static Pods. Their manifests live in /etc/kubernetes/manifests/

```
# staticPodPath in kubelet config:
staticPodPath: /etc/kubernetes/manifests
# View static pods in kube-system:
$ kubectl --namespace kube-system get pods -o wide

# Delete a static pod:
$ sudo rm /etc/kubernetes/manifests/<pod.yaml>
```

	Static Pods	DaemonSets
Created by	kubelet	kube-apiserver
Use case	Control plane components	Monitoring/logging agents
kube-scheduler	Ignored by both	

● Autoscaling -- HPA & VPA

Horizontal Pod Autoscaler (HPA) -- Automatically scales out (adds/removes replicas) based on CPU utilization, memory, or custom metrics. Well-suited for stateless workloads.

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: php-apache
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: php-apache
  minReplicas: 3
  maxReplicas: 20
  metrics:
    - type: Resource
  resource:
    name: cpu
  target:
    type: Utilization
    averageUtilization: 50
```

HPA uses the Metrics Server to collect resource data, then calculates the number of replicas needed based on current vs. target utilization.

Vertical Pod Autoscaler (VPA) -- Automatically scales up/down the resource requests and limits of containers in a pod. Best for stateful workloads.

```
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
  name: php-apache
spec:
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: php-apache
  updatePolicy:
    updateMode: "Auto"
  resourcePolicy:
    containerPolicies:
      - containerName: "*"
  minAllowed:
    cpu: 10m
  memory: 5Mi
  maxAllowed:
    cpu: 200m
  memory: 500Mi
```

VPA updateMode	Behavior
Auto	Automatically updates resource requests/limits based on usage.
Off	Manual mode -- only provides recommendations, no automatic updates.

● ConfigMap & Secrets

ConfigMap -- Stores non-sensitive configuration data as key-value pairs. Reference in deployments via environment variables or file mounts.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-configmap
data:
  DB_HOST: "mydbhost"
  DB_PORT: "5432"
  DB_NAME: "mydb"
config.yaml: |
  setting1: value1
  setting2: value2
```

3 Ways to Use ConfigMap

Step 1: Environment Variables -- `envFrom.configMapRef`

Step 2: Volume Mounts -- Mount as files in container at `/etc/config`

Step 3: Command-line Args -- Inject values into command arguments

Secrets -- Similar to ConfigMaps but for sensitive data (passwords, tokens, API keys). Values are base64-encoded. Note: base64 is encoding, not encryption!

```
apiVersion: v1
kind: Secret
metadata:
  name: db-secrets
type: Opaque
data:
  DB_USER: dXNlcg== # base64
  DB_PASSWORD: cGFzc3dvcmQ= # base64
# Create from command line:
$ kubectl create secret generic db-secret \
  --from-literal=username=myuser \
  --from-literal=password=mypass

# Decode a secret value:
$ kubectl get secret db-secret -o jsonpath=\
  '{.data.password}' | base64 --decode
```

■■ base64 is not encryption! Use external key management (HashiCorp Vault, Azure Key Vault, AWS KMS) for true secret encryption at rest.

Secret Type	Use Case
Opaque	Default -- arbitrary data
kubernetes.io/tls	TLS certificates & keys
docker-registry	Docker registry credentials
ssh-auth	SSH private/public keys
service-account	Auto-created for service accounts

● Init Container

An Init Container runs before the main containers in a pod. Used for setup tasks: database schema setup, downloading files, waiting for a service to become available. If an init container fails, k8s restarts it until it succeeds.

```
apiVersion: v1
kind: Pod
metadata:
  name: my-webapp-pod
spec:
  initContainers:
    - name: redis-setup
      image: redis:latest
      command: ["sh", "-c"]
      args:
        - |
          redis-cli ping || exit 1
          redis-cli config set maxmemory 1gb
          redis-cli config set maxmemory-policy allkeys-lru
  containers:
    - name: webapp
      image: my-webapp-image
      env:
        - name: REDIS_HOST
          value: redis-service
```

Step P: Pending -- Pod accepted by k8s; scheduler assigning to node; images being pulled.

Step R: Running -- At least one container is running, starting, or restarting.

Step S: Succeeded -- All containers terminated successfully. Won't restart.

Step F: Failed -- All containers terminated; at least one exited with failure.

Step U: Unknown -- State cannot be determined (usually node communication issue).

Pod execution order: Init Container → Sidecar Container → Main Container

● Sidecar Container & Jobs

Sidecar Container -- Deployed alongside the main container in a pod. Keeps the main container focused on its task while delegating auxiliary tasks (logging, monitoring, backups) to the sidecar. They share the same network and storage.

Common use cases:

Logging & Monitoring: Collect and forward logs/metrics from the main container to a central system.

Backup & Recovery: Perform periodic backup operations on the main container's data.

Service Mesh: Implement a service mesh (Istio, Linkerd) to manage and secure communications between services.

Job -- Creates one or more pods to run a finite batch process. Tracks successful completions. When specified completions reached, job is complete. Used for data processing, backups, migrations.

```
apiVersion: batch/v1
kind: Job
metadata:
  name: data-processing-job
spec:
  backoffLimit: 3
template:
  spec:
    containers:
      - name: data-processor
image: data-processor:v1.4
command: ["python", "process.py"]
restartPolicy: Never
```

CronJob: Higher-level abstraction that schedules Jobs on a cron schedule. schedule: "0 * * * *" runs at top of every hour. Completed Jobs are retained by default -- set successfulJobsHistoryLimit to auto-clean.

Strategy	Description
RollingUpdate	Gradually replaces old pods with new ones. Zero downtime. Default strategy.
Recreate	Deletes all old pods before creating new ones. Causes brief downtime.
Blue/Green	Runs both old (blue) and new (green) versions simultaneously. Switch traffic after verification.
Canary	Updates a small percentage of pods first to test the new version before full rollout.

```
# Create deployment
$ kubectl create deployment nginx \
  --image=nginx:1.17 --replicas=3

# Update image (triggers rollout)
$ kubectl set image deployment/nginx \
  nginx=nginx:1.18

# Check rollout status
$ kubectl rollout status deployment/nginx

# Pause a rollout
$ kubectl rollout pause deployment/nginx

# Rollback to previous version
$ kubectl rollout undo deployment/nginx

# View revision history
$ kubectl rollout history deployment nginx

# Rollback to specific revision
$ kubectl rollout undo deployment/nginx \
  --to-revision=2
```

● Probes (Self-Healing)

Probes play a vital role in checking the health and availability of pods and containers. k8s provides three types of probes:

■ **Liveness Probe** -- Determines when to restart a container. Detects deadlocks or frozen apps. If liveness probe fails, kubelet kills and restarts the container.

■ **Readiness Probe** -- Determines when container is ready to accept traffic. Pod removed from Service endpoints while readiness probe fails. No restart -- just traffic routing.

■ **Startup Probe** -- Used for slow-starting containers. Disables liveness & readiness probes until startup probe succeeds. Prevents premature restarts on startup.

HTTP Check

```
livenessProbe:
  httpGet:
    path: /healthz
  port: 8080
  initialDelaySeconds: 30
  periodSeconds: 10
```

HTTP GET to the path. Response 200–399 = success. Any other code = failure.

TCP Check

```
livenessProbe:
  tcpSocket:
    port: 8080
  initialDelaySeconds: 15
  periodSeconds: 10
  failureThreshold: 3
```

Tries to open TCP connection on the port. Connection established = success. Otherwise = failure.

Exec Check

```
livenessProbe:
  exec:
    command:
      - /bin/sh
      - -c
      - /usr/bin/check.sh
  initialDelaySeconds: 30
  periodSeconds: 10
```

Executes command inside container. Exit code 0 = success. Non-zero = failure. failureThreshold: consecutive failures before action.

● Node Maintenance

Node maintenance involves temporarily taking a node out of the cluster to apply updates, patches, or replace hardware. During maintenance, workloads are evicted and rescheduled.

```
$ kubectl cordon kubeworker-1      # mark unschedulable
$ kubectl drain kubeworker-1 --ignore-daemonsets # evict pods
# perform maintenance here...
$ kubectl uncordon kubeworker-1    # re-enable scheduling
```

Version Policy: k8s supports the last 3 minor versions for 9 months. Always upgrade one minor version at a time (1.25 → 1.26, not 1.25 → 1.27). Component Version Skew: • Control plane components: 0 versions (identical) • kubelet/kubectl: up to 2 minor versions behind • etcd: up to 1 minor version behind API server

Step 1: Upgrade Control Plane

```
# Drain control plane node
$ kubectl drain <cp-node> --ignore-daemonsets
# Upgrade kubeadm
$ sudo apt-mark unhold kubeadm
$ sudo apt-get upgrade -y kubeadm=1.26.7-00
$ sudo apt-mark hold kubeadm

# Plan the upgrade
$ sudo kubeadm upgrade plan

# Apply upgrade
$ sudo kubeadm upgrade apply v1.26.7

# Upgrade kubelet + kubectl, restart
$ sudo apt-get upgrade -y kubelet=1.26.7-00 kubectl=1.26.7-00
$ sudo systemctl restart kubelet
$ kubectl uncordon <cp-node>
```

Step 2: Upgrade Worker Nodes

```
# For each worker node:
$ kubectl drain <worker-node> --ignore-daemonsets
$ apt-mark unhold kubeadm && apt-get install -y kubeadm=1.26.7-00
$ sudo kubeadm upgrade node
$ apt-mark unhold kubelet kubectl \
  && apt-get install -y kubelet=1.26.7-00 kubectl=1.26.7-00
$ sudo systemctl restart kubelet
$ kubectl uncordon <worker-node>
```

Worker Upgrade Strategy	Risk
All at once	■ High -- all nodes offline simultaneously
+1/-1 (add new, remove old)	■ Medium -- minimal downtime
-1/+1 (remove old, add new)	■ Medium-high -- brief resource shortage

Worker Upgrade Strategy	Risk
One at a time	■ Low -- safest approach

Backup etcd Before Upgrade

```
$ ETCDCTL_API=3 etcdctl snapshot save snapshot.db \  
  --endpoints=$ENDPOINTS \  
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \  
  --cert=/etc/kubernetes/pki/etcd/server.crt \  
  --key=/etc/kubernetes/pki/etcd/server.key
```


● Authentication & Authorization

Every API request goes through: Authentication → Authorization → Admission Control → etcd

Authentication Methods

Service Account Tokens -- Auto-created JWT tokens for each namespace. Used by pods to authenticate to the API. From k8s 1.24+, tokens must be explicitly requested via TokenRequest API or Secret.

X.509 Client Certificates -- Each client has a certificate & private key signed by the cluster CA. Commonly used for admin users. Strong authentication method.

Webhook Authentication -- External HTTP service verifies identity. Useful for integrating custom auth systems (e.g., LDAP, OIDC).

Control access to resources based on user roles. Define Roles, then bind them to users/groups/service accounts via RoleBindings.

Object	Scope	Description
Role	Namespace	Permissions within a single namespace
ClusterRole	Cluster	Permissions across entire cluster
RoleBinding	Namespace	Binds Role to user/group/SA in namespace
ClusterRoleBinding	Cluster	Binds ClusterRole to user/group/SA globally

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
namespace: dev
rules:
  - apiGroups: [""]
    resources: ["pods"]
    verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: pod-reader-binding
namespace: dev
subjects:
  - kind: User
    name: john
apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: pod-reader
apiGroup: rbac.authorization.k8s.io
```

● Service Account

Service Accounts are non-human accounts providing unique identities for pods and system components. Every namespace has a default service account. From k8s 1.24+, tokens must be explicitly created via the TokenRequest API or Secret objects.

Step 1: Create Service Account `kubectl create serviceaccount monitoring-agent-SA`

Step 2: Grant Permissions via ClusterRole + ClusterRoleBinding

Step 3: Mount in Pod using serviceAccountName field

```
# Create + bind a service account
$ kubectl create serviceaccount sara
$ kubectl create role pod-reader \
  --verb=get,list,watch --resource=pods
$ kubectl create rolebinding pod-reader-binding \
  --role=pod-reader --serviceaccount=default:sara

# Get a token for a service account
$ kubectl -n default create token sara

# Check SA permissions
$ kubectl auth can-i --list \
  --as=system:serviceaccount:default:sara
```

Mounting SA in a Pod

```
apiVersion: v1
kind: Pod
metadata:
  name: monitoring-agent
spec:
  serviceAccountName: monitoring-agent-SA
containers:
  - name: monitoring-agent
image: monitoring-agent-image
volumeMounts:
  - name: sa-token
mountPath: /var/run/secrets/kubernetes.io/serviceaccount
readOnly: true
volumes:
  - name: sa-token
secret:
  secretName: monitoring-agent-SA-token
```

The token is mounted at `/var/run/secrets/kubernetes.io/serviceaccount/token` and can be used to authenticate API calls from within the pod.

● API Groups

k8s API groups organize related resources and operations. Core group (/api/v1) contains fundamental resources. Named groups (/apis/{group}/{version}) extend functionality.

API Group	Resources
core (v1)	Pods, Services, Namespaces, ConfigMaps, Secrets, PV, PVC, Nodes
apps	Deployment, ReplicaSet, StatefulSet, DaemonSet
batch	Job, CronJob
rbac.authorization.k8s.io	Role, ClusterRole, RoleBinding, ClusterRoleBinding
networking.k8s.io	NetworkPolicy, Ingress
storage.k8s.io	StorageClass, VolumeAttachment, CSINode

API Versions: v1alpha1 -- unstable, may change significantly v1beta1 -- well-tested, enabled by default, may still change v1 -- stable, production-ready

```
# List all API groups and versions
$ kubectl api-versions

# List all API resources
$ kubectl api-resources

# List namespaced resources
$ kubectl api-resources --namespaced=true

# List cluster-scoped resources
$ kubectl api-resources --namespaced=false
```

Scope	Examples
Namespaced	Pods, Services, Deployments, ConfigMaps, Roles, RoleBindings, PVCs
Cluster-scoped	Nodes, PersistentVolumes, ClusterRoles, ClusterRoleBindings, Namespaces

● Authentication with X.509 (Creating a User)

How to create a new admin/developer user account for k8s cluster access using X.509 certificates:

Step 1: Generate private key & CSR `$ openssl genrsa -out mojtaba.key 2048` `$ openssl req -new -key mojtaba.key -subj "/CN=mojtaba" -out mojtaba.csr`

Step 2: Create CertificateSigningRequest YAML and submit `apiVersion: certificates.k8s.io/v1 kind: CertificateSigningRequest metadata: name: mojtaba spec: request: $(cat mojtaba.csr | base64 -w 0) signerName: kubernetes.io/kube-apiserver-client usages: [client auth]`

Step 3: Approve the CSR `$ kubectl get csr` `$ kubectl certificate approve mojtaba` `$ kubectl get csr mojtaba -o jsonpath='{.status.certificate}' | base64 -d > mojtaba.crt`

Step 4: Create kubeconfig for the user `$ kubectl config set-cluster cka --server=https://kubemaster-1:6443 --certificate-authority=ca.crt --embed-certs` `$ kubectl config set-credentials mojtaba --client-key=/path/mojtaba.key --client-certificate=/path/mojtaba.crt` `$ kubectl config set-context mojtaba@cka --cluster=cka --user=mojtaba --namespace=dev` `$ kubectl config use-context mojtaba@cka`

Step 5: Set up RBAC for the user `$ kubectl create role developer --verb=list,get,create,update,delete --resource=pods --namespace=dev` `$ kubectl create rolebinding developer-binding --role=developer --user=mojtaba --namespace=dev`

● Auditing

Auditing records API requests and changes in the cluster for security, compliance, and troubleshooting.

Audit Level	What's Logged
None	Do not log any events.
Metadata	Log request metadata only (who, what, when, where).
Request	Log metadata + request content (excludes response).
RequestResponse	Log metadata + request content + response content.

```
# Add to kube-apiserver spec.containers.command:
- --audit-policy-file=/etc/kubernetes/audit/policy.yaml
- --audit-log-path=/var/log/kubernetes/audit/audit.log
- --audit-log-maxsize=500
- --audit-log-maxbackup=3
```

Define rules for ingress and egress traffic between pods. By default all traffic is allowed. After a NetworkPolicy is applied, only explicitly allowed traffic is permitted.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: db-policy
spec:
  podSelector:
    matchLabels:
      role: db
  policyTypes: [Ingress]
  ingress:
    - from:
        - podSelector:
            matchLabels:
              role: django
  ports:
    - protocol: TCP
      port: 3306
```

■■ Network Policies require a CNI plugin that supports them (e.g., Calico, Weave Net).

Defines privilege and access control settings for a Pod or container: file system permissions, user IDs, Linux capabilities.

```
spec:
  securityContext:      # Pod-level
  runAsUser: 1000
  runAsGroup: 2000
  fsGroup: 3000
  readOnlyRootFilesystem: true
  containers:
    - name: my-container
  securityContext:      # Container-level
  capabilities:
    add: [NET_ADMIN]
    drop: [CHOWN]
    allowPrivilegeEscalation: false
```

Gatekeeper (OPA)

Kubernetes admission controller leveraging Open Policy Agent (OPA). Enforces custom policies across the cluster via Constraint Templates and Constraints (CRDs).

ConstraintTemplate: Defines the schema and Rego logic for the policy. Constraint: Instance of a ConstraintTemplate applied to specific resources.

Step →: User creates/updates resource → request goes to Gatekeeper (admission webhook) → checks all defined Constraints → rejects if any policy is violated.

● Storage in Kubernetes

HostPath Volume -- Mounts a directory from the host node's filesystem into the pod. Good for testing/development. Not recommended for production -- creates security risks and node dependency.
volumes: - name: user-uploads hostPath: path: var/uploads type: DirectoryOrCreate

EmptyDir Volume -- Temporary storage created when pod starts, deleted when pod stops. Shared between containers in same pod. Can store data in memory (medium: Memory) for fast caching. volumes: - name: cache-volume emptyDir: medium: Memory sizeLimit: 1Gi

ConfigMaps & Secrets -- Can be mounted as volumes in a Pod, allowing the Pod to access the data as files. Config files, environment data, and secrets can all be treated as file mounts.

PVs are cluster-wide storage resources provisioned by admins. PVCs are requests for storage by pods. PVs and PVCs exist independently of any individual pod.

Access Mode	Description
ROX	ReadOnlyMany -- multiple nodes, read-only
RWX	ReadWriteMany -- multiple nodes, read-write
RWO	ReadWriteOnce -- single node, read-write

Reclaim Policy	Behavior when PVC is deleted
Retain	PV & data preserved; manual cleanup required
Delete	PV & underlying storage deleted
Recycle	Deprecated -- basic scrub & make available again


```
## Persistent Volume (PV)
kind: PersistentVolume
apiVersion: v1
metadata:
  name: nfs-pv1-40g-rw
spec:
  capacity:
    storage: 40Gi
accessModes:
  - ReadWriteMany
nfs:
  server: nfs_server_ip
path: /mnt/nfs_share/pv1-40g-rw
persistentVolumeReclaimPolicy: Retain
---
## Persistent Volume Claim (PVC)
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: nfs-pvc-20g
spec:
  accessModes:
    - ReadWriteMany
resources:
  requests:
    storage: 15Gi
---
## Pod using PVC
volumes:
  - name: my-volume
persistentVolumeClaim:
  claimName: nfs-pvc-20g
```

Dynamic Provisioning (StorageClass)

Instead of manually creating PVs, a StorageClass allows k8s to automatically create PVs when a PVC is created.

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: gce-pd-storage
provisioner: kubernetes.io/gce-pd
parameters:
  type: pd-standard
replication-type: none
reclaimPolicy: Retain
allowVolumeExpansion: true
volumeBindingMode: Immediate
```

● StatefulSet

StatefulSets manage stateful applications requiring unique identities, stable network addresses, persistent storage, ordered deployment and scaling. Unlike Deployments, each pod maintains a sticky identity -- even when rescheduled.

Predictable Pod Nameswebapp-0, webapp-1, webapp-2

Fixed DNS Nameswebapp-0.hs-web.default.svc.cluster.local

Ordered CreationEach pod waits for previous to be ready before starting.

Headless ServiceAccompanies each StatefulSet for direct pod communication.

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mysql-hs
spec:
  serviceName: mysql # mandatory - headless svc
  replicas: 3
  selector:
    matchLabels:
      app: mysql
  template:
    metadata:
      labels:
        app: mysql
    spec:
      containers:
        - name: mysql
      image: mysql:latest
      volumeMounts:
        - name: mysql-shared-storage
      mountPath: /var/lib/mysql
      volumeClaimTemplates: # dedicated PVC per pod
        - metadata:
            name: mysql-shared-storage
      spec:
        accessModes: ["ReadWriteMany"]
        storageClassName: google-storage
      resources:
        requests:
          storage: 10Gi
```

A Headless Service has no ClusterIP (clusterIP: None). Instead of load balancing, it returns the IP addresses of all matching pods directly. Essential for StatefulSets -- allows direct pod-to-pod communication using stable DNS names.

```
apiVersion: v1
kind: Service
metadata:
  name: mysql-hs
spec:
  ports:
    - port: 3306
  selector:
    app: mysql
  clusterIP: None # makes it headless
```

	Regular Service	Headless Service
Has ClusterIP	■ Yes	■ No (None)
Load Balancing	■ Yes	■ No
DNS returns	Single VIP	All pod IPs
Used with	Stateless apps	StatefulSets, DBs

Shared Storage: All pods share same PVC. Good for caches, tmp files. Risk of data corruption if multiple pods write same files.

Dedicated Storage: Each pod gets own PVC via volumeClaimTemplates. Best for databases, unique files.

● Kustomize

Kustomize manages k8s manifests using a declarative approach. Define base manifests and apply environment-specific overlays without modifying the base files.

```
# kustomization.yaml (base)
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - deployment.yaml
  - service.yaml
---
# overlay kustomization.yaml (dev)
bases:
  - ../../base
nameSuffix: "-dev"
patchesStrategicMerge:
  - patch.yaml # adds replica: 1
# Apply with kustomize:
$ kubectl apply -k ./overlays/dev
```

Helm is the package manager for Kubernetes. It packages k8s resources as charts -- reusable, versioned bundles. Supports rollbacks, dependencies management, and value templating.

Chart structure: Chart.yaml (metadata), values.yaml (defaults), templates/ (YAML templates), helpers.tpl (reusable snippets)

```
# Add a Helm chart repository
$ helm repo add bitnami https://charts.bitnami.com/bitnami
$ helm repo update

# Search for charts
$ helm search repo wordpress

# Install a chart
$ helm install my-wordpress bitnami/wordpress \
  --namespace wordpress -f values.yaml

# Upgrade a release
$ helm upgrade my-wordpress bitnami/wordpress \
  -f values2.yaml

# Rollback to previous version
$ helm rollback my-wordpress 1 -n wordpress

# List releases
$ helm list -n wordpress

# Get custom values for a release
$ helm get values my-wordpress --revision=2
```

An Operator is a method of packaging, deploying, and managing a specific application using Kubernetes controllers. Operators automate complex operational tasks via custom resources (CRDs) and custom controllers.

A database operator might automate: provisioning new instances, scaling, backups/restores, failovers, migrations -- tasks traditionally requiring human expertise.

Step 1: Developer defines a Custom Resource (CR) specifying desired state (e.g., kind: MyApp)

Step 2: Custom Controller watches for CR changes and reconciles actual state to desired state

Step 3: Operator manages underlying StatefulSets, Services, PVCs automatically

● Ingress

Ingress is an API object that allows external access to services in a cluster. It provides Layer 7 load balancing, SSL termination, and name-based virtual hosting. A separate Ingress Controller (e.g., Nginx) must be deployed to implement Ingress rules.

Ingress vs LoadBalancer: LoadBalancer Service requires one public IP per service. Ingress exposes multiple services through a single IP address using path/host-based routing.

Path-Based Routing Example

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: ingress-path
spec:
  rules:
    - host: shop.com
  http:
    paths:
      - path: /video
        pathType: Prefix
        backend:
          service:
            name: video-service
    paths:
      - path: /wear
        pathType: Prefix
        backend:
          service:
            name: wear-service
  port:
    name: http
    number: 80
```

How Ingress Controller Works

Step 1: Define an Ingress Resource with routing rules.

Step 2: Ingress Controller watches for new/modified Ingress resources, Services, and EndpointSlices.

Step 3: Controller configures load balancer (e.g., generates nginx.conf) to implement desired routing.

Customizing Nginx Ingress

Helm Chart Values: Override settings during chart installation.

ConfigMap: Global Nginx configuration (log format, timeouts, GeoIP).

Annotations: Per-ingress-rule configuration.

```
# Basic auth annotation example:
annotations:
  nginx.ingress.kubernetes.io/auth-type: basic
  nginx.ingress.kubernetes.io/auth-secret: shop-basic-auth
  nginx.ingress.kubernetes.io/auth-realm: Authentication Required
```

● Cert-Manager

Cert-Manager is a certificate management controller for k8s. It automates issuance and renewal of TLS certificates from Let's Encrypt, HashiCorp Vault, Venafi, and more.

Main features: • Issuing & renewing certificates from various sources • Automated creation/updating of k8s Secrets with certificates • DNS01 and HTTP01 ACME challenge solvers • Issuing certificates for Ingress resources via annotations

Step 1: Create a ClusterIssuer

```
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: letsencrypt-prod
spec:
  acme:
    server: https://acme-v02.api.letsencrypt.org/directory
    email: [email protected]
    privateKeySecretRef:
      name: letsencrypt-prod
  solvers:
    - http01:
        ingress:
          class: nginx
```

Step 2: Create a Certificate Resource

```
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: arye-ir-cert
spec:
  secretName: arye-ir-tls
  issuerRef:
    name: letsencrypt-prod
    kind: ClusterIssuer
  dnsNames:
    - arye.ir
    - *.arye.ir
  duration: 90d
```

Step 3: Configure Ingress to Use TLS


```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: arye.ir-ingress
annotations:
  nginx.ingress.kubernetes.io/ssl-redirect: "true"
# Or use annotation-based cert-manager:
cert-manager.io/cluster-issuer: "letsencrypt-prod"
spec:
  tls:
    - hosts:
        - arye.ir
        - *.arye.ir
  secretName: arye-ir-tls
  rules:
    - host: arye.ir
  http:
    paths:
      - path: /
    pathType: Prefix
  backend:
    service:
      name: your-service
  port:
    number: 80

```

Step 4: Verify Certificate

```

$ kubectl describe certificate arye-ir-cert
$ kubectl describe secret arye-ir-tls

```

Issuer Types

Issuer Type	Use Case
ACME	Let's Encrypt & other CAs. Automates certificate issuance via HTTP01 or DNS01 challenge.
CA	Existing certificate authority. Provide CA cert & private key.
SelfSigned	Self-signed certificates. Good for testing/development only.

Annotations vs Certificate Resources: • Use annotations for basic, single TLS certificate configuration (simpler, less flexible). • Use Certificate resources for multiple certs, automation, and advanced management (more complex, more powerful).

● Backup & Restore

Cluster Configuration Backup -- Kubernetes manifests: Store all YAML files in Git (deployments, services, configmaps). Version control provides history and rollback capability. etcd data: Back up using etcdctl snapshots or tools like Velero.

Backup etcd with etcdctl

```
$ ETCDCTL_API=3 etcdctl snapshot save snapshot.db \
  --endpoints=$ENDPOINTS \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key
```

Restore etcd Backup

```
# 1. Stop kube-apiserver
$ sudo mv /etc/kubernetes/manifests/kube-api.yaml ~/

# 2. Restore snapshot
$ etcdctl snapshot restore snapshot.db \
  --data-dir=/var/lib/etcd-from-backup

# 3. Update etcd manifest to use new data dir
# Edit /etc/kubernetes/manifests/etcd.yaml
# Change hostPath to: /var/lib/etcd-from-backup
# 4. Restart kube-apiserver
$ sudo mv ~/kube-api.yaml /etc/kubernetes/manifests/
```

Application Data Backup -- Backup strategies for application data:

Database Backups: Use database-native tools (mysqldump, mongodump). Schedule via CronJobs.

Backup Sidecars: Deploy a sidecar container in pods dedicated to periodic backups to remote storage (S3, GCS).

Volume Snapshots: k8s VolumeSnapshot CRDs create point-in-time copies of PV content without creating new volumes.

Backup with Velero

```
# Create a full cluster backup
$ velero backup create <backup-name>

# Backup specific namespaces only
$ velero backup create <backup-name> \
  --include-namespaces my-namespace

# Restore from a backup
$ velero restore create \
  --from-backup <backup-name>

# Restore specific resources only
$ velero restore create \
  --from-backup <backup-name> \
  --include-resources deployments,pods
```

● Quick Reference -- kubectl Cheat Sheet

Pods

```
$ kubectl get pods -o wide
$ kubectl describe pod <name>
$ kubectl logs <pod> [-c <container>]
$ kubectl exec -it <pod> -- /bin/bash
$ kubectl delete pod <name>
$ kubectl top pods
```

Deployments

```
$ kubectl get deployments
$ kubectl scale deployment <name> --replicas=5
$ kubectl set image deployment/<name> <c>=<img>
$ kubectl rollout status deployment/<name>
$ kubectl rollout undo deployment/<name>
```

Services & Networking

```
$ kubectl get services
$ kubectl get endpoints
$ kubectl port-forward pod/<name> 8080:80
$ kubectl expose deployment <name> \
  --port=80 --type=NodePort
```

ConfigMaps & Secrets

```
$ kubectl create configmap <name> \
  --from-literal=key=value
$ kubectl create secret generic <name> \
  --from-literal=pass=secret
$ kubectl get secrets
$ kubectl describe configmap <name>
```

Nodes & Cluster

```
$ kubectl get nodes -o wide
$ kubectl describe node <name>
$ kubectl cordon <node>
$ kubectl drain <node> --ignore-daemonsets
$ kubectl uncordon <node>
$ kubectl top nodes
```

RBAC

```
$ kubectl get roles,rolebindings
$ kubectl auth can-i create pods
$ kubectl auth can-i list pods --as=user
$ kubectl get clusterroles
```

Pro Tips: | Use `-o yaml` to see the full resource manifest | Use `--dry-run=client -o yaml` to generate YAML without applying | Use `kubectl explain <resource>` to see field documentation | Use

● Additional API Resource Types

The following Kubernetes resource types complete the full API surface. These resources cover admission control, traffic management, dynamic resource allocation, flow control, and CSI storage drivers.

ReplicationController (rc) — [apps/v1](#) [legacy]

ReplicationController (RC) -- The original way to ensure a specified number of Pod replicas are running at all times. Mostly superseded by ReplicaSet and Deployment in modern Kubernetes. Selector only supports equality-based matching (no set-based). Still supported for backward compatibility.

```
# Create a ReplicationController
apiVersion: v1
kind: ReplicationController
metadata:
  name: nginx-rc
spec:
  replicas: 3
  selector:
    app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.25
          ports:
            - containerPort: 80

# Get RC status
$ kubectl get rc
$ kubectl describe rc nginx-rc
# Scale RC
$ kubectl scale rc nginx-rc --replicas=5
```

MutatingWebhookConfiguration — [admissionregistration.k8s.io/v1](#)

MutatingWebhookConfiguration -- Defines admission webhooks that can mutate (modify) API objects before they are persisted. Useful for injecting sidecars, setting defaults, or enforcing organizational policies on resource creation/modification.

```
apiVersion: admissionregistration.k8s.io/v1
kind: MutatingWebhookConfiguration
metadata:
  name: sidecar-injector
webhooks:
- name: sidecar.example.com
  admissionReviewVersions: ["v1"]
  clientConfig:
    service:
      name: sidecar-injector-svc
      namespace: default
      path: "/mutate"
    caBundle: <base64-ca-cert>
  rules:
  - operations: ["CREATE"]
    apiGroups: [""]
    apiVersions: ["v1"]
    resources: ["pods"]
  sideEffects: None
  failurePolicy: Ignore

# List all mutating webhooks
$ kubectl get mutatingwebhookconfigurations
$ kubectl describe mutatingwebhookconfiguration sidecar-injector
```

ValidatingAdmissionPolicy — admissionregistration.k8s.io/v1 [k8s 1.26+]

ValidatingAdmissionPolicy -- CEL (Common Expression Language)-based validation policies introduced in Kubernetes 1.26 (GA in 1.30). Replaces complex webhook setups for common validation rules. Bound to resources via ValidatingAdmissionPolicyBinding. Much lighter than full webhook servers.

```
# Policy: Enforce image registry
apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingAdmissionPolicy
metadata:
  name: enforce-registry
spec:
  failurePolicy: Fail
  matchConstraints:
    resourceRules:
      - apiGroups:  [""]
        apiVersions: ["v1"]
        operations: ["CREATE", "UPDATE"]
        resources:  ["pods"]
  validations:
    - expression: |
        object.spec.containers.all(c,
          c.image.startsWith("registry.company.com/"))
      message: "Images must be from registry.company.com"
---
# Binding: attach policy to all namespaces
apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingAdmissionPolicyBinding
metadata:
  name: enforce-registry-binding
spec:
  policyName: enforce-registry
  validationActions: [Deny]
  matchResources: {}

$ kubectl get validatingadmissionpolicies
$ kubectl get validatingadmissionpolicybindings
```

ValidatingWebhookConfiguration — admissionregistration.k8s.io/v1

ValidatingWebhookConfiguration -- Defines admission webhooks that validate API objects without modifying them. If any validating webhook denies the request, the operation fails. Used for policy enforcement, compliance checks, and security controls.


```
apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingWebhookConfiguration
metadata:
  name: policy-enforcer
webhooks:
- name: validate.example.com
  admissionReviewVersions: ["v1"]
  clientConfig:
    service:
      name: policy-server
      namespace: default
      path: "/validate"
  rules:
  - operations: ["CREATE", "UPDATE"]
    apiGroups: ["apps"]
    apiVersions: ["v1"]
    resources: ["deployments"]
  sideEffects: None
  failurePolicy: Fail
  namespaceSelector:
    matchLabels:
      environment: production

$ kubectl get validatingwebhookconfigurations
# Temporarily disable (for emergencies only):
$ kubectl delete validatingwebhookconfiguration policy-enforcer
```

CustomResourceDefinition (CRD) — [apiextensions.k8s.io/v1](#)

CustomResourceDefinition -- Extends the Kubernetes API with custom resource types. CRDs are the foundation of the Operator pattern. Once created, custom resources behave like native k8s resources and can be managed with kubectl. Used by Helm, cert-manager, Istio, Prometheus Operator, and virtually every major k8s add-on.

```

apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: databases.example.com
spec:
  group: example.com
  versions:
    - name: v1
      served: true
      storage: true
      schema:
        openAPIV3Schema:
          type: object
          properties:
            spec:
              type: object
              properties:
                engine:
                  type: string
                  enum: [postgres, mysql, mongodb]
                replicas:
                  type: integer
                  minimum: 1
  scope: Namespaced
  names:
    plural: databases
    singular: database
    kind: Database
    shortNames: [db]

# After creating CRD, use custom resource:
$ kubectl get databases
$ kubectl apply -f my-database.yaml
$ kubectl get crds # list all CRDs

```

ControllerRevision — apps/v1 [internal / informational]

ControllerRevision -- Stores an immutable snapshot of the state of a controller (DaemonSet or StatefulSet) at a particular point in time. Automatically created by Kubernetes for DaemonSets and StatefulSets to support rolling updates and rollbacks. Not typically created manually -- managed by the system.

```

# List controller revisions (auto-created by k8s)
$ kubectl get controllerrevisions -n <namespace>

# Revisions for a specific DaemonSet
$ kubectl get controllerrevisions \
  -l app=my-daemonset

# View revision details
$ kubectl describe controllerrevision <name>

# Rollback uses these revisions internally:
$ kubectl rollout undo daemonset/my-ds
$ kubectl rollout undo statefulset/my-sts --to-revision=2

```

IngressClass — `networking.k8s.io/v1`

IngressClass -- Represents a class of Ingress controllers. Allows multiple Ingress controllers to run in the same cluster. The Ingress resource references an IngressClass via the `ingressClassName` field. One IngressClass can be marked as the default for the cluster.

```
apiVersion: networking.k8s.io/v1
kind: IngressClass
metadata:
  name: nginx
  annotations:
    # Mark as cluster default:
    ingressclass.kubernetes.io/is-default-class: "true"
spec:
  controller: k8s.io/ingress-nginx
  parameters:
    apiGroup: k8s.example.com
    kind: IngressParameters
    name: external-lb
---
# Reference in Ingress:
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: my-ingress
spec:
  ingressClassName: nginx # references IngressClass
  rules: []

$ kubectl get ingressclasses
$ kubectl describe ingressclass nginx
```

IPAddress & ServiceCIDR — `networking.k8s.io/v1` [k8s 1.31+]

IPAddress -- Represents a single IP address allocation within the cluster. Part of the new IP Address Management (IPAM) feature in Kubernetes 1.31+. **ServiceCIDR** defines the IP range from which ClusterIP addresses are assigned to Services. These resources provide visibility into IP allocation and support multiple service CIDR ranges.

```
# ServiceCIDR defines the pool for ClusterIP services
apiVersion: networking.k8s.io/v1
kind: ServiceCIDR
metadata:
  name: primary
spec:
  cidrs:
    - 10.96.0.0/16
    - fd00:1::/112

# IPAddress (auto-managed by k8s)
# Lists all allocated IPs
$ kubectl get ipaddresses

# List Service CIDRs
$ kubectl get servicecidrs

# View IP allocations for services
$ kubectl get ipaddresses \
  -o wide

# Note: IPAddress/ServiceCIDR require
# the MultiCIDRServiceAllocator feature gate
```

ResourceClaim & DeviceClass — resource.k8s.io/v1 [Dynamic Resource Allocation]

Dynamic Resource Allocation (DRA) -- Kubernetes 1.26+ feature for requesting specialized hardware resources (GPUs, FPGAs, custom ASICs). ResourceClaim describes a resource request by a Pod. DeviceClass defines a category of devices available in the cluster. ResourceClaimTemplate creates ResourceClaims per-Pod from a template.

```
# DeviceClass: defines a category of hardware
apiVersion: resource.k8s.io/v1
kind: DeviceClass
metadata:
  name: gpu.nvidia.com
spec:
  selectors:
    - cel:
        expression: |
          device.driver == "nvidia.com/gpu"
---
# ResourceClaim: request a specific device
apiVersion: resource.k8s.io/v1
kind: ResourceClaim
metadata:
  name: my-gpu-claim
spec:
  devices:
    requests:
      - name: gpu
        deviceClassName: gpu.nvidia.com
        allocationMode: ExactCount
        count: 1
---
# Pod using a ResourceClaim
apiVersion: v1
kind: Pod
spec:
  resourceClaims:
    - name: gpu
      resourceClaimName: my-gpu-claim
  containers:
    - name: app
      resources:
        claims:
          - name: gpu

$ kubectl get resourceclaims
$ kubectl get deviceclasses
```

CSIDriver — storage.k8s.io/v1

CSIDriver -- Describes a Container Storage Interface (CSI) Driver installed in the cluster. Allows CSI drivers to customize Kubernetes behavior: ephemeral inline volumes, pod info passthrough, volume attachment skipping, and storage capacity tracking. Automatically created when a CSI driver is installed (e.g., aws-ebs-csi-driver, gcp-pd-csi-driver).

```
apiVersion: storage.k8s.io/v1
kind: CSIDriver
metadata:
  name: ebs.csi.aws.com
spec:
  # Allow volumes to be attached without a node
  attachRequired: true
  # Pass pod info to CSI driver
  podInfoOnMount: true
  # Driver supports volume expansion
  requiresRepublish: false
  # Supported volume lifecycle modes
  volumeLifecycleModes:
    - Persistent
    - Ephemeral
  # Enable storage capacity tracking
  storageCapacity: true

# List all CSI drivers in cluster
$ kubectl get csidrivers
$ kubectl describe csidriver ebs.csi.aws.com

# List CSI nodes (per-node driver info)
$ kubectl get csinodes

# Check storage capacity
$ kubectl get csistoragecapacities -A
```

FlowSchema & PriorityLevelConfiguration — flowcontrol.apiserver.k8s.io/v1

API Priority and Fairness (APF) -- Controls how the API server handles concurrent requests.

FlowSchema classifies incoming API requests into flows. PriorityLevelConfiguration defines limits and queuing behavior for each priority level. Prevents any single client from overwhelming the API server.

```
# PriorityLevelConfiguration: set limits
apiVersion: flowcontrol.apiserver.k8s.io/v1
kind: PriorityLevelConfiguration
metadata:
  name: custom-high-priority
spec:
  type: Limited
  limited:
    nominalConcurrencyShares: 30
    limitResponse:
      type: Queue
      queuing:
        queues: 64
        handSize: 6
        queueLengthLimit: 50
---
# FlowSchema: classify requests
apiVersion: flowcontrol.apiserver.k8s.io/v1
kind: FlowSchema
metadata:
  name: monitoring-flows
spec:
  priorityLevelConfiguration:
    name: custom-high-priority
  matchingPrecedence: 500
  distinguisherMethod:
    type: ByUser
  rules:
    - subjects:
        - kind: ServiceAccount
          serviceAccount:
            name: prometheus
            namespace: monitoring
      resourceRules:
        - apiGroups: ["*"]
          resources: ["*"]
          verbs: ["get", "list", "watch"]

$ kubectl get flowschemas
$ kubectl get prioritylevelconfigurations
```

Summary: All API Resource Types at a Glance

Resource	API Group	Namespaced	Purpose
ReplicationController	v1 (core)	Yes	Legacy pod replica management
MutatingWebhookConfig	admissionregistration.k8s.io	No	Mutate objects before persist
ValidatingAdmissionPolicy	admissionregistration.k8s.io	No	CEL-based validation (1.26+)
ValidatingWebhookConfig	admissionregistration.k8s.io	No	Validate objects via webhook

Resource	API Group	Namespaced	Purpose
CustomResourceDefinition	apiextensions.k8s.io	No	Extend k8s API with custom types
ControllerRevision	apps/v1	Yes	Snapshot for rollback (DS/STS)
IngressClass	networking.k8s.io	No	Define Ingress controller class
IPAddress	networking.k8s.io	No	Track IP allocations (1.31+)
ServiceCIDR	networking.k8s.io	No	Define ClusterIP address range
ResourceClaim	resource.k8s.io	Yes	Request specialized hardware (DRA)
DeviceClass	resource.k8s.io	No	Define hardware device categories
CSIDriver	storage.k8s.io	No	Register CSI storage drivers
FlowSchema	flowcontrol.apiserver.k8s.io	No	Classify API request flows (APF)
PriorityLevelConfig	flowcontrol.apiserver.k8s.io	No	Set API concurrency limits (APF)